

Computer Engineering Department

King Mongkut's University of Technology Thonburi

CPE 241 Database

Problem Session #07 Queries for backend processing

Date: Check PS07 assessment activities on LEB2

Introduction

In this lab, after refining your database design with business requirements and applying proper normalization, you will shift your focus to the reporting needs of administrative users. These reports are intended to generate meaningful insights and analytics that support data-driven decision-making. You will design queries that involve data aggregation and time-based analysis, enabling administrators to interpret trends, monitor performance, and evaluate system activities effectively.

Task List

- 1. Design Reports for Admin Users**
- 2. Develop Backend Queries for Report Generation**

Task 1: Design Reports for Admin Users

In this task, students will identify and design meaningful analytical reports that support administrative decision-making within their selected project domain. Reports must go beyond simple data retrieval and instead demonstrate analytical thinking through aggregation, time-based trends, and multi-table relationships.

Objectives of Task 1

1. To understand how database systems support managerial and strategic decision-making.
2. To translate business requirements into analytical reporting requirements.
3. To design report structures involving multi-table joins and aggregation.
4. To practice presenting structured data in visual and interpretable formats.

Instructions

- **Quantity Requirement**
The total number of reports required for each group must equal the number of group members multiplied by two. In other words, **each group member is required to propose at least two administrative reports** relevant to the project topic. The proposed reports should address realistic decision-support needs within the context of the system being developed and demonstrate clear analytical value for administrative users.
- **Complexity Requirement**
Within each group, at least **three reports** must involve:
 - Joins across 3–4 related tables,
 - At least one aggregation function, and
 - A time-based analysis component (e.g., grouped by day, month, quarter, or year).
- **Example Report Types**
 - Transaction or booking statistics grouped by category and month.
 - Most accessed entities within a given time period.
 - Revenue or usage growth analysis across quarters.

- **Report Documentation**

For each report, you must clearly describe

- The business question it answers.
- The tables involved and the attributes used.
- Create visual outputs for all designed reports with mockup data. These may include structured tables or charts. Visualizations should clearly reflect the analytical objective of each report and be properly labeled (title, axis labels, units, legends if necessary).

Deliverables

1. List of all proposed administrative reports.
2. Clearly indicate at least three reports that satisfy the complexity requirement.
3. Report documentation for each report.

Submission Format

- Compile all written explanations, pictures, and tables into a single PDF report.
- Organize the report with clear sections.
- Include captions for each picture and table.

Task 2: Develop Backend Queries for Report Generation

In this task, students will implement SQL queries that generate the data required for the reports designed in Task 1. Queries must correctly retrieve, join, aggregate, and format data to support the required visualizations.

Objectives of Task 2

1. To apply SQL techniques including multi-table joins and aggregation.
2. To implement time-based grouping and sorting logic.
3. To ensure query correctness, efficiency, and readability.
4. To bridge database querying with reporting and analytics requirements.

Instructions

STEP 1: Query Implementation

- For each designed report, write the corresponding SQL query.
- Use clear indentation and readable SQL formatting.
- Provide comments explaining complex logic.
- Example queries include:

```
-- Count the number of students registered per department
SELECT departmentName, COUNT(studentID) AS total_students
FROM student
JOIN department ON student.departmentID = department.departmentID
GROUP BY departmentName;
```

```
-- Get the number of course enrollments per month
SELECT MONTH(registration_date) AS month, COUNT(enrollmentID) AS
total_enrollments
FROM enrollment
GROUP BY month;
```

STEP 2: Query Validation

- Test each query with sample data.
- Ensure the results match the intended business question.
- Verify correctness of aggregation results.

Deliverables

1. All SQL queries used to generate each report.
2. Screenshots or exported results of query outputs.

Submission Format

- Compile all written explanations and pictures into a single report.
-

Final Submission

- Combine results from Task 1 and Task 2 in a PDF file and ensure it is named appropriately (e.g., **PS07_YourGroupName.pdf**) then submit the report on LEB2 before the deadline.
- Upload the PDF report to the Assessment Activities section for PS07 on LEB2.